

03

코드 리뷰 Agent

Codex CLI 기반 설계·구현·개선

차성재 · Agentic AI PM / AI 겸임교수

5주 과정의 현재 위치

주차	실행할 서비스	이번 주 활용
1주차	Tool Calling과 실행 환경	함수·입력 검증·Test
2주차	한국어 회의기록 Agent	Context·사람 검토·화면
3주차	코드 리뷰 Agent	오류 재현·Codex 리뷰·수정
4주차	GitHub PR 자동 리뷰	PR 수집·댓글·문서 연동
5주차	업무 서비스 통합·미니 프로젝트	운영·개선·개인 서비스

지난 수업과 오늘의 확장

1·2주차 활용 기반

- 입력을 공통 형식으로 정리
- LLM 결과를 근거와 대조
- 사람 확인 후 다음 단계

3주차의 새 학습

- Git Diff와 코드 라인 해석
- 버그 재현과 회귀 테스트
- 리뷰 품질의 전후 비교

원본 모듈과 운영 순서

원본 모듈	배정 시간	5주 운영에서의 위치
Agent·환경 / Tool Calling	6H / 6H	1·2주차 기초
코드 리뷰 Agent	8H	3주차 집중 구현
GitHub 자동 리뷰	6H	4주차 1~6차시
회의록·문서 자동화	8H	2주차 선행 + 4·5주차 통합
통합·운영·프로젝트	6H	5주차 3~8차시

장바구니 오류의 전후

입력은 같고 결제 금액은 다릅니다.

수정 전

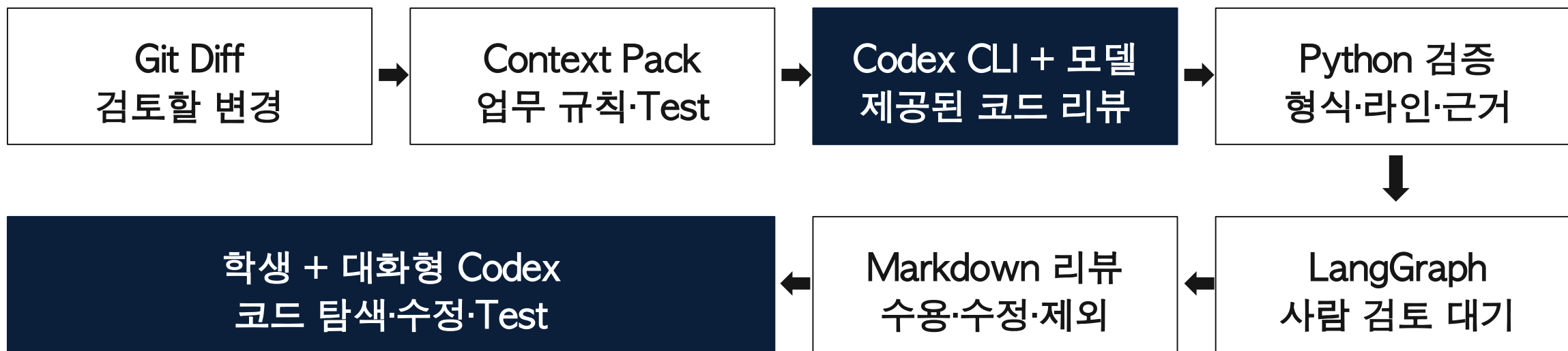
-2,000원

수정 후

→ 3,000원

상품 10,000원 / 쿠폰 15,000원
할인은 상품 금액까지만, 배송비는 별도

코드 리뷰 Agent 전체 구조



반복 수정 Diff와 테스트 결과로 다시 검토

4주차 GitHub PR 리뷰

5주차 업무 통합·개인 서비스

자동화 구성의 Role

구성	직접 수행하는 일	판단 기준
Codex 대화 / Adapter	대화는 탐색·수정 / Adapter는 전달 자료 검토	업무 규칙과 관련 코드
Python	입력 검증·CLI 실행·Test 확인	명시적 함수와 결과 계약
LangGraph	검토 대기·수정·재개	사람의 선택과 실행 상태
사람	지적 수용·수정 범위·최종 반영	영향과 재현 증거

오전 수업과 점심시간

시간	구분	함께 챙길 내용
09:00-09:50	1차시	리뷰 대상과 업무 규칙
09:50-10:40	2차시	Git Diff와 재현 Test
10:40-11:30	3차시	리뷰 기준과 Context
11:30-12:00	쉬는 시간	오전 수업 후 30분
12:00-13:00	점심시간	13시 수업 시작

오후 수업과 Q&A

시간	구분	함께 챙길 내용
13:00-14:40	4·5차시	CLI 연동·코드 수정
14:40-15:00	쉬는 시간	앞선 2차시 후 20분
15:00-17:30	6·7·8차시	검토 흐름·품질·서비스
17:30-18:00	쉬는 시간·Q&A	마지막 30분·실습 복구

시작 전 준비

준비물	확인 명령·위치	완료 표시
Python 3.12 권장	<code>python --version</code>	선택한 가상환경 경로
VS Code + Jupyter	Notebook 우측 Kernel 선택	코드 셀 실행 가능
Codex CLI	<code>codex --version</code> <code>codex login status</code>	설치 버전·로그인 방식
Git	<code>git status --short</code>	현재 저장소·변경 파일

3주차 코드 다운로드

```
# 처음 다운로드: macOS Terminal / Git Bash
git clone --branch codex/day3-review-intelligence \
  https://github.com/smilesjcha/llm-agent-and-workflow-automation.git
cd llm-agent-and-workflow-automation
git branch --show-current
```

3주차 배포: Draft PR #1의 작업 branch / 기존 폴더: git status로 수정 파일 확인 / 기존 수정이 있으면 별도 폴더·ZIP 사용

Python과 Notebook 시작

```
python -m venv .venv
# macOS: source .venv/bin/activate
# Windows PowerShell: .venv\Scripts\Activate.ps1
python -m pip install -r requirements-day3.txt
jupyter lab materials/day3/
# day3_review_intelligence_lab.ipynb 열기
```

- 실행 위치: 저장소 루트
- Kernel: 방금 설치한 .venv
- 설치 셀은 Notebook 상단에도 포함

Codex CLI 실행 조건

학생 PC

- codex 명령과 Python 실행
- ChatGPT 로그인 상태 재사용
- 수업용 코드와 Test 파일

연결된 모델 서비스

- 리뷰 판단과 응답 생성
- 인터넷·계정 이용 권한 필요
- 이용 한도 도달 시 Fixture 재생

3주차 1차시

09:00-09:50

리뷰 대상과 업무 규칙

장바구니 결제 오류 재현

이론 15분 · 시연 8분 · 코드 실습 22분 · 확인 5분

코드 리뷰의 판단 대상

코드의 모양보다 사용자에게 생기는 문제

- 같은 입력에서 결과가 잘못되는가
- 허용하지 않은 요청을 처리하는가
- 실패가 성공처럼 보이는가

교육용 쇼핑 정책

조건	기대 동작	확인 입력
쿠폰 상한	상품 금액을 넘는 할인 금지	10,000원 / 쿠폰 15,000원
배송비 기준	할인 후 50,000원 이상 무료	50,000원 / 쿠폰 10,000원
입력 범위	금액은 0 이상의 정수	음수·문자열·True
영수증	상품·할인·배송·결제 금액 일치	합계 다시 계산

계산 순서의 차이

수정 전

- 원금으로 무료 배송 판단
- 쿠폰을 제한 없이 차감
- 음수 입력도 계산 진행

기대 동작

- 쿠폰 금액 상한 적용
- 할인 후 배송 조건 확인
- 잘못된 입력은 명확한 오류

할인 후 금액과 최종 결제

```
# 생성된 starter 폴더에서 실행
from checkout import payable, calculate_checkout

print(payable(10_000, 15_000))
receipt = calculate_checkout(10_000, 15_000)
print(receipt["shipping_won"])
print(receipt["payable_won"])
```

- payable: 상품 잔액
-5,000원
- 배송비: 3,000원
- 최종 결제: -2,000원

첫 실행과 오류 재현

01 Notebook 1차시에서 실습 폴더 생성

02 starter의 calculate_checkout 실행

03 동일 입력으로 solution 결과 비교

10,000 / 15,000: -2,000 → 3,000 · 50,000 / 10,000: 40,000 → 43,000

예상값의 근거

테스트 정답은 업무 규칙에서 계산

- 할인 후 상품 금액: 0원
- 무료 배송 기준 미달: 3,000원
- 최종 결제: 3,000원

직접 구현: 정상 주문과 초과 쿠폰

```
def learner_payable(total_won, coupon_won):  
    return total_won - coupon_won  
  
assert learner_payable(30_000, 5_000) == 25_000  
assert learner_payable(10_000, 15_000) == -5_000  
# 두 번째 assert는 오류 재현 확인입니다.
```

- Notebook 1차시의 직접 작성 함수
- 정상 입력과 초과 쿠폰을 대조
- 쿠폰 상한 수정은 5차시에서 적용

Codex 대화: 변경 전 분석

요청

교육용 장바구니 코드를 읽어 주세요.
requirements.md의 할인·배송 정책과 비교해
사용자에게 금액 오류가 생기는 입력을 두 개 제시해 주세요.
아직 코드를 수정하지 말고 계산 과정을 설명해 주세요.

응답 확인 재현 입력과 실제 값 · 위반한 정책 · 지적인 코드 위치

리뷰 지적의 구성

필드	학생에게 필요한 설명
위치	어느 파일의 몇 번째 줄인가
영향	어떤 입력에서 사용자가 손해를 보는가
근거	업무 규칙·코드·실행 결과 중 무엇인가
교정	최소한으로 무엇을 바꿀 것인가

지적과 스타일 제안

고쳐야 할 동작

- 쿠폰 초과로 음수 결제
- 할인 전 금액으로 배송비 결정

선택 가능한 스타일

- 변수 이름의 취향
- 같은 의미의 문장 순서

입력 한 개의 추가 실험

01 쿠폰을 0원·같은 금액·초과 금액으로 변경

02 예상값을 먼저 메모

03 실행값과 차이가 나는 조건 설명

계산 순서를 직접 설명 · 실패 입력을 다시 실행

3주차 2차시

09:50-10:40

Git Diff와 재현 테스트

변경 줄과 실패原因的 연결

이론 13분 · 시연 7분 · 코드 실습 25분 · 확인 5분

Git Diff의 정보

변경된 코드와 검토 위치의 연결

- 이전 파일과 새 파일의 차이
- 추가·삭제·유지 줄의 구분
- 리뷰는 변경 후 라인 번호 기준

Diff 기호와 줄 번호

기호	의미	새 파일의 줄 번호
+	추가한 줄	증가
-	삭제한 줄	그대로
공백	유지한 문맥	증가
@@	변경 구간 Hunk	새 구간의 시작값

Hunk의 시작점

```
@@ -4,2 +4,3 @@
  def payable(total, coupon):
-     return total - coupon
+     discounted = total - min(total, coupon)
+     return discounted
```

- -4: 이전 파일 시작 위치
- +4: 새 파일 시작 위치
- 추가 줄:
새 파일 5번·6번

직접 구현: 새 줄 번호 계산

```
hunk_lines = [" keep", "-old", "+new"]
new_line, added = 2, []
for line in hunk_lines:
    if line.startswith("+"):
        added.append((new_line, line[1:]))
        new_line += 1
    elif line.startswith(" "):
        new_line += 1
assert added == [(3, "new")]
```

- 한 Hunk의 줄 번호 계산 예제
- 전체 함수: Notebook 2차시
- 삭제 줄은 새 번호를 소비하지 않음

두 구간의 라인 복원

01 Notebook 2차시 multi-hunk 입력 열기

02 learner_added_line_map 함수 작성

03 예상 라인과 실제 결과 assert 비교

첫 Hunk의 추가 줄 · 두 번째 Hunk의 시작값 유지

리뷰 위치의 실패

잘못된 매핑

- 삭제 줄을 새 번호로 계산
- 두 번째 Hunk에서도 이전 번호 유지
- 존재하지 않는 999번 줄

학생 확인

- 변경 후 파일을 직접 열기
- Diff 헤더에서 시작값 확인
- 추가 줄 집합에 속하는지 검사

실제 Test 실행

```
# 생성된 starter 폴더에서 실행
python checkout_checks.py

# 저장소 루트에서 Agent component 확인
python -m pytest -q \
    tests/test_day3_review_copilot.py
```

- starter 실패:
재현하려는 업무 오류
- Agent component
통과: 리뷰 도구 검증
- 두 테스트 대상은 서로
다름

실패 테스트의 읽기 순서

입력·예상값·실제값·호출 위치

- AssertionError가 가리키는 비교
- 예상값은 정책에서 계산
- 첫 원인과 연쇄 실패 구분

Codex 대화: 재현 테스트

요청

쿠폰 초과 적용 오류를 재현하는 Test를 추가해 주세요.
기대값은 requirements.md에서 계산해 주세요.
starter 코드의 동작은 아직 바꾸지 마세요.
실행 명령과 실패 이유를 함께 설명해 주세요.

응답 확인 새 테스트가 실제 실패 · 기대값의 정책 근거 · 구현 변경이 없는 Diff

실패 한 건의 최소 재현

01 실패 항목 하나를 선택

02 입력 두 개와 예상값만 남긴 테스트 작성

03 동일 테스트를 두 번 실행

항상 같은 실패 재현 · 실패 이유를 한 문장으로 설명

실행 오류와 제품 오류

화면의 신호	가능한 원인	다음 확인
ModuleNotFoundError	Kernel·라이브러리 불일치	sys.executable과 설치 셀
파일 없음	실행 폴더 불일치	Path.cwd()와 파일 경로
AssertionError	정책과 계산 결과 불일치	실제값·기대값

3주차 3차시

10:40-11:30

리뷰 기준과 Context

업무 규칙을 포함한 리뷰 요청

이론 15분 · 시연 7분 · 코드 실습 23분 · 확인 5분

리뷰 Context의 내용

판단에 필요한 코드와 업무 규칙

- 변경 코드만으로 알 수 없는 제품 정책
- 함수의 호출 관계와 관련 Test
- 검토 범위와 제외 항목

두 요청의 정보 차이

짧은 요청

- 이 코드 리뷰해 줘
- 파일 내용만 제공

업무 맥락 포함

- 쿠폰 상한·할인 후 배송 정책
- 재현 입력·관련 Test·변경 줄
- 근거와 최소 교정 요구

최소 Context Pack

정보	이 실습의 예시	선택 이유
업무 규칙	requirements.md	정답의 근거
변경 코드	checkout.py의 Diff	지적할 위치
관련 테스트	checkout_checks.py	이미 확인한 동작
리뷰 기준	AGENTS.md / Task	어떤 문제를 찾을지

직접 구현: 허용 정보 선택

```
def learner_public_context(source):  
    allowed = ("business_rules",  
              "changed_lines", "test_evidence")  
    return {key: source[key] for key in allowed  
            if key in source}  
  
review_context = learner_public_context(context_source)  
assert "unrelated_note" not in review_context
```

Notebook 3차시 context_source 사용 / 업무 규칙·변경 줄·Test만 포함 / 관계없는 필드의 제외도 assert 확인

Prompt의 네 가지 구성

목표·맥락·판단 기준·응답 형식

- 목표: 결제 오류와 입력 검증 리뷰
- 맥락: 할인·배송 정책과 테스트
- 판단 기준: 재현 가능한 결함과 사용자 영향
- 응답 형식: 위치·영향·근거·최소 수정

Codex 대화: 리뷰 요청 V1

요청

결제 코드를 리뷰해 주세요.

응답 확인

찾은 문제 · 빠진 업무 규칙 · 불필요한 스타일 제안

Codex 대화: 리뷰 요청 V2

요청

requirements.md와 변경 Diff를 기준으로 검토해 주세요.
쿠폰 상한, 할인 후 무료배송, 음수·문자열 입력을 확인해 주세요.
실제 추가 줄을 근거로 영향·재현 입력·최소 수정을 적어 주세요.
확인하지 않은 실행 결과는 단정하지 마세요.

응답 확인 정책을 인용한 지적 · 실제 줄과 일치하는 위치 · 재현 가능한 입력

AGENTS.md와 이번 Task

지속적인 프로젝트 기준

- 허용 파일 범위
- Test·오류 처리 규칙
- 리뷰 답변 형식

이번 변경의 목표

- 쿠폰 계산 오류 한 개 수정
- 관련 테스트만 추가
- 작은 Diff로 검토 요청

Review Finding의 구조

```
# 실제 starter 결함의 주요 필드 발췌
finding = {
    "path": "checkout.py", "line": 5,
    "severity": "P1", "title": "쿠폰 상한 누락",
    "impact": "초과 쿠폰에서 상품 잔액이 음수",
    "evidence": "return total_won - coupon_won",
    "correction": "할인을 상품 금액으로 제한",
}
```

- 원본 starter 5번 줄의 실제 코드
- 전체 계약에는 rule_id 등도 포함
- 사용자는 Markdown 리뷰로 확인

LangChain Prompt Template

```
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages([
    ("system", "코드 변경과 실제 Test를 근거로 검토하는 리뷰어입니다."),
    ("human", "{review_request}"),
])
messages = template.invoke(
    {"review_request": refined_prompt}).to_messages()
assert messages[1].content == refined_prompt
```

Notebook 3차시 refined_prompt 사용 / system Role과 human 요청을 분리 / 입력 바인딩
단계: 아직 모델 호출 없음

Context를 포함한 두 요청

- 01 같은 Diff로 baseline-refined 요청 작성
- 02 refined에 업무 규칙과 Test 근거 추가
- 03 Markdown 요청 파일의 포함 정보 비교

prompt_baseline.md·prompt_refined.md · 실제 응답 비교는 4차시 CLI 연결 후

Context의 과잉과 누락

상황	리뷰에 생기는 문제	교정
업무 정책 없음	잘못된 기대값	정책 문서 추가
전체 저장소 복사	관련 없는 코드에 집중	변경 주변과 관련 Test 선택
실행 결과 없음	추정과 사실 혼동	Test 종료 코드 제공

3주차 4차시

13:00-13:50

Codex CLI 연동

Python에서 실제 Codex 리뷰 실행

이론 12분 · 시연 8분 · 코드 실습 25분 · 확인 5분

Codex CLI의 두 사용 방식

대화로 개발하고 Python에서 리뷰 호출

- 대화형: 분석·구현·테스트 요청
- 비대화형 exec: 프로그램에서 한 작업 실행
- 동일한 프로젝트 규칙과 리뷰 기준 재사용

CLI 설치와 로그인 확인

```
# Node.js 설치 후, Codex가 없을 때만
npm install -g @openai/codex
codex --version
codex login
codex login status
codex exec --help
```

- 이미 로그인된 PC: status부터 확인
- 로그인 화면: 본인 ChatGPT 계정
- 확인: codex 명령을 같은 Terminal에서 실행

로컬 CLI와 API 직접 호출

오늘의 Codex CLI

- ChatGPT 로그인 활용
- 프로젝트를 읽는 Coding Agent
- codex exec를 Python에서 실행

직접 OpenAI API

- API key로 요청 인증
- 앱이 도구 실행과 상태를 설계
- 별도 API 과금·권한 적용

한 번의 리뷰 요청

```
# Notebook이 출력한 EXERCISE 폴더에서 실행
codex exec --sandbox read-only \
  --ephemeral --color never \
  "requirements.md와 starter/checkout.py를 검토하고
  재현 입력과 최소 수정 방안을 설명해 주세요."
```

- 실행 위치:
requirements.md가
있는 폴더
- read-only: 리뷰
과정의 파일 수정 제한
- 모델: CLI에서 사용
가능한 기본 모델

Python의 CLI 호출 원리

```
import subprocess

completed = subprocess.run(
    ["codex", "exec", "--sandbox", "read-only", "-"],
    input=refined_prompt, text=True,
    capture_output=True, timeout=180,
    cwd=EXERCISE,
)
print(completed.returncode)
```

호출 원리 예제: 실제 서비스는 Adapter 사용 / stdin: Prompt를 명령과 분리 / timeout: 기다릴 수 있는 시간 제한

응답 처리의 순서

단계	확인 항목	실패 시
프로세스	종료 코드·시간 초과	오류 코드 반환
응답 형식	필수 Field·자료형	Schema 오류
리뷰 근거	추가 줄과 실제 내용	후보 제외
실행 경로	codex_cli / fixture	사용 경로 표시

실제 Codex 리뷰 실행

01 Notebook 4차시에서 RUN_CODEX_LIVE=True

02 CodexCLIReviewProvider로 리뷰 실행

03 Provider·오류·Markdown 지적 확인

provider_used=codex_cli · 실제 업무 규칙과 지적 대조

대화형 Agent와 리뷰 Adapter

도구 사용과 고정된 리뷰 호출의 구분

- 대화형 Codex: 코드 탐색·수정·Test
- 수업용 Adapter: 제공된 Context만 검토
- Python: 형식·라인·오류를 일관되게 검증

Codex 대화: Adapter 구현

요청

Python에서 codex exec를 호출하는 리뷰 함수를 만들어 주세요.
입력은 Diff와 업무 규칙, 출력은 ReviewFinding 목록입니다.
시간 초과·실행 실패·응답 형식 오류를 구분해 주세요.
성공과 실패 Test를 추가하고 실제 명령을 실행해 주세요.

응답 확인 명령 인자와 stdin · 안정적인 오류 코드 · Mock 테스트와 실제 실행 구분

CLI 실행 문제의 복구

문제	확인	다음 동작
명령을 못 찾음	codex --version	Terminal 재시작·PATH 확인
로그인 필요	codex login status	codex login
시간 초과·한도	오류 코드와 계정 상태	입력 축소·대기·Fixture 선택
응답 형식 오류	Schema와 실제 Field	요청 수정 후 재실행

Live 응답과 Fixture 재생

Live Codex

- 지금 실제 모델이 생성한 리뷰
- 계정·네트워크 영향
- 응답 지연과 내용이 변할 수 있음

Fixture 복구

- 제공된 응답을 읽어 후속 과정 재현
- 모델 호출 없음
- 실제 모델 품질 평가로 사용하지 않음

3주차 5차시

13:50-14:40

수정 코드와 회귀 테스트

리뷰 근거를 코드 수정으로 반영

이론 10분 · 시연 8분 · 코드 실습 27분 · 확인 5분

리뷰에서 수정까지

재현 입력이 교정의 검증 기준

- 지적인 오류를 먼저 재현
- 관련 계산만 작게 수정
- 기존 기능이 유지되는지 재확인

교정할 계산과 입력 조건

오류	최소 수정	확인 입력
초과 쿠폰	<code>min(total_won, coupon_won)</code>	10,000 / 15,000
무료 배송 기준	할인 후 payment로 조건 계산	50,000 / 10,000
잘못된 입력	정수·0 이상 검사	-1 / 소수 / True

입력 검증의 함정

`isinstance(value, int)`

- 정수 값을 검사
- Python에서 True도 통과

`type(value) is int`

- 정확히 int만 허용
- True·문자열을 분리
- 금액 정책에 맞는 선택

직접 구현: 입력 검사

```
def validate_money(value):  
    if isinstance(value, bool) or not isinstance(value, int):  
        raise ValueError("MONEY_INTEGER_REQUIRED")  
    if value < 0:  
        raise ValueError("MONEY_NON_NEGATIVE_REQUIRED")  
  
validate_money(0)  
# True·10_000.5·-1은 각각 오류 확인
```

Notebook 5차시 수정 파일의 함수 / Test가 기대하는 오류 코드를 유지 / 정상 입력은 예외 없이 다음 계산 진행

직접 구현: 할인과 배송비

```
def payable(total_won, coupon_won):  
    validate_money(total_won)  
    validate_money(coupon_won)  
    return total_won - min(total_won, coupon_won)  
  
# calculate_checkout 내부의 배송비 계산  
payment = payable(10_000, 15_000)  
shipping = 0 if payment >= 50_000 else 3_000  
assert payment == 0  
assert payment + shipping == 3_000
```

- 앞 장표의 `validate_money`부터 실행
- `payable`은 배송비 전 상품 잔액 반환
- Notebook 5차시 REPAIRED_SOURCE 발췌

Codex 대화: 작은 수정

요청

쿠폰 상한과 할인 후 배송비 오류를 수정해 주세요.
requirements.md를 기준으로 관련 함수만 바꾸어 주세요.
음수·문자열·True 입력 Test도 추가해 주세요.
Test를 통과시키기 위해 기대값을 낮추지 마세요.

응답 확인 정책과 일치하는 구현 · 기존 Test 유지 · 수정 범위와 실제 결과

Codex 수정 실행 기록

관찰 대상	실제로 확인한 결과
변경 범위	checkout.py 1개 수정 checkout_checks.py 유지
같은 테스트	수정 전 2개 통과·7개 실패 수정 후 9개 통과
명령 오류 복구	python 명령 없음 확인 python3로 다시 실행
구현 선택	실제 할인액 = total_won - payment 참고 구현의 min과 결과 대조

수정 파일의 재실행

- 01 생성된 starter/checkout.py 수정
- 02 checkout_checks.py 실행
- 03 실패한 입력을 영수증 함수로 다시 실행

의도한 실패가 사라짐 · 9개 업무 Test와 영수증 일치

수정 전후의 계산 결과

상품 50,000원 / 쿠폰 10,000원

할인 전 배송 기준

40,000원

할인 후 배송 기준

→ 43,000원

40,000원 상품 잔액 + 3,000원 배송비

직접 구현: 근거 없는 후보 제외

```
def learner_grounded_candidates(
    candidates, added_lines,
):
    valid = {(x["path"], x["line"]) for x in added_lines}
    kept, removed = [], []
    for item in candidates:
        key = (item["path"], item["line"])
        target = kept if key in valid else removed
        target.append(item)
    return kept, removed
```

Notebook의 learner_lines를 두 번째 인자로 전달 / 유지 후보와 제외 후보를 함께 반환 / 999번 줄 후보를 넣어 제외 결과 확인

회귀 Test와 AI 리뷰

회귀 Test

- 정해진 입력과 기대값 비교
- 반복 실행 결과가 일정
- 명확한 계약 오류 탐지

AI 리뷰

- 업무 맥락과 설계 영향 검토
- 아직 Test가 없는 후보 제안
- 실행 근거와 사람이 재검증

추가 조건 한 개의 구현

01 무료 배송 임계값 근처 입력 선택

02 49,999·50,000·50,001원 Test 작성

03 Codex 수정과 본인 계산 결과 비교

경계값 세 개의 계산 · 기존 기능 유지 확인

3주차 6차시

15:00-15:50

LangGraph 검토 흐름

사람 결정에 따른 실행 재개

이론 15분 · 시연 7분 · 코드 실습 23분 · 확인 5분

사람 검토가 필요한 지점

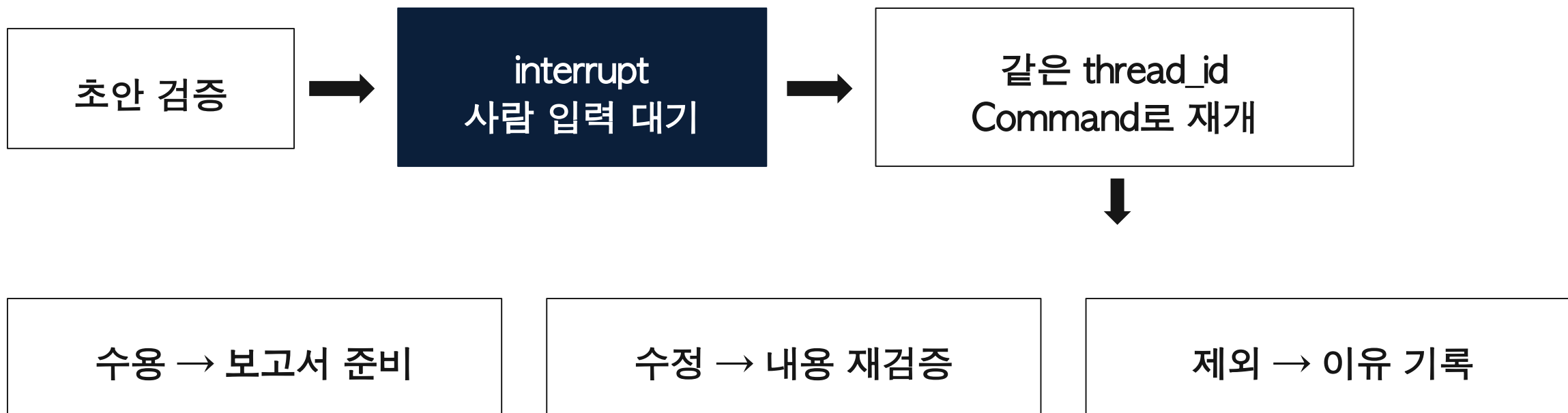
리뷰 초안과 최종 수정의 구분

- 근거가 부족한 지적은 제외
- 맞는 지적도 수정 범위 판단
- 사람 결정 전 다음 단계 대기

LangGraph 구성 요소

용어	이 서비스의 의미
State	Diff·Finding·사람 결정·Test 결과
Node	검토·검증·판정 함수
Edge	다음 함수로 이동하는 경로
Checkpoint	실행 중 상태를 저장하는 지점

검토 대기와 실행 재개



미결정은 승인 아님 · 실제 GitHub 게시와 별도 단계

직접 구현: 검토자 입력 검증

```
def learner_review_decision(decision, reviewer, rationale):  
    if decision not in {"approve", "edit", "reject"}:  
        raise ValueError("REVIEW_DECISION_INVALID")  
    if not reviewer.strip() or not rationale.strip():  
        raise ValueError("REVIEW_REASON_REQUIRED")  
    return {"decision": decision,  
            "reviewer": reviewer, "rationale": rationale}
```

Notebook 6차시의 실제 검증 함수 / 선택·검토자·이유를 함께 전달 / 미결정은 interrupt 상태에서 대기

interrupt와 resume

```
# Notebook 6차시: graph_draft와 graph_config 준비 후
review_graph = build_review_graph()
graph_start = review_graph.invoke(
    {"draft": graph_draft, "audit": []}, config=graph_config)
assert "__interrupt__" in graph_start

resume_payload = learner_review_decision(
    "approve", "수강생", "재현 입력과 Test 확인")
graph_final = review_graph.invoke(
    Command(resume=resume_payload), config=graph_config)
```

build_review_graph 내부의 interrupt에서 대기 / 필드명은 choice가 아닌 decision / 같은 graph_config로 원래 실행 재개

수용·수정·제외

선택	학생이 확인할 것	다음 상태
수용	재현 입력·근거·교정 타당성	보고서 준비
수정	수정한 지적의 위치·필수 Field	재검증
제외	과잉 지적·다른 정책 적용 여부	이유 기록
미결정	아직 검토하지 않은 항목	대기

실제 LangGraph 중단 실험

- 01 Notebook 6차시 Graph compile
- 02 첫 invoke에서 interrupt 확인
- 03 각 선택은 새 Graph로 시작·같은 thread로 재개

중단 전후 State 변화 · 거절 시 후속 전달 보류

Codex 대화: 검토 노트

요청

리뷰 초안 뒤에 LangGraph Human Review를 추가해 주세요.
검토자가 수용·수정·제외를 선택하고 이유를 남깁니다.
미결정은 대기 상태로 유지해 주세요.
같은 thread에서 재개되는 Test를 추가해 주세요.

응답 확인 상태 저장과 thread_id · 분기별 테스트 · 실제 게시와 분리된 결과 준비

Workflow와 Agent의 역할

Workflow에 고정된 순서

- Diff 파싱
- 응답 Schema 검증
- 사람 결정에 따른 이동

Agent가 판단하는 부분

- 읽을 관련 코드 선택
- 문제 후보와 재현 입력 추론
- 수정 전략 제안

재개와 재실행의 차이

같은 실행의 맥락 유지

- 재개: 저장된 상태에서 사람 답변 전달
- 재실행: 처음부터 새로운 입력 처리
- 실행 식별자로 두 흐름 구분

수정된 지적의 검증

01 REVIEW_EDITED_FINDINGS의 제목 직접 수정

02 REVIEW_DECISION="edit"로 같은 실행 재개

03 추가 실험: 새 Graph에 999번 줄 수정안 전달

수정한 제목이 최종 보고서에 반영 · 추가 실험의 잘못된 위치는 BLOCKED

3주차 7차시

15:50-16:40

리뷰 품질과 개선 비교

오탐·누락·수정 효과 측정

이론 12분 · 시연 8분 · 코드 실습 25분 · 확인 5분

리뷰 품질의 측정

유효한 지적과 놓친 오류

- 오탐: 문제 없는 코드를 문제로 지적
- 누락: 실제 오류를 찾지 못함
- 고친 뒤 지적이 사라지는지도 확인

Precision과 Recall

지표	질문	계산
Precision	지적 중 실제 문제가 몇 개인가	$TP / (TP + FP)$
Recall	실제 문제 중 몇 개를 찾았는가	$TP / (TP + FN)$
F1	오탐과 누락을 함께 보면 어떤가	두 지표의 조화평균

Notebook의 평가 예제

계산 학습용 Baseline · 실제 모델 점수와 별도

올바른 지적

1

과잉 지적

1

놓친 문제

1

Precision 0.5 / Recall 0.5 / F1 0.5

직접 구현: 지적과 정답의 비교

```
def learner_review_metrics(predicted, expected):  
    p, e = set(predicted), set(expected)  
    tp, fp, fn = len(p & e), len(p - e), len(e - p)  
    precision = tp / (tp + fp) if tp + fp else 0.0  
    recall = tp / (tp + fn) if tp + fn else 0.0  
    denom = precision + recall  
    f1 = 2 * precision * recall / denom if denom else 0.0  
    return {"tp": tp, "fp": fp, "fn": fn, "f1": f1,  
            "precision": precision, "recall": recall}
```

Notebook과 같은 두 집합 입력·dict 반환 / TP: 공통 결함 / FP: 과잉 / FN: 누락 / 실제 Codex 지적은 사람이 결함에 연결

Golden Set의 구성

유형	준비할 코드	확인 목적
명확한 오류	쿠폰 상한·배송 조건 누락	오류 탐지
수정한 코드	같은 정책의 solution	불필요한 지적 억제
위험한 동작	eval·shell·인증정보 로그	보안 규칙
실패 입력	경로 이탈·형식 오류	실행 중단과 오류 코드

비교 실험의 변수

고정할 조건

- 동일 코드와 업무 규칙
- 같은 평가 기준
- 같은 입력 사례

바꿀 조건

- Prompt의 정책 설명
- 읽을 Context 범위
- 수정 전·후 코드 중 하나

Prompt와 코드 개선 비교

- 01 V1·V2의 지적을 평가표에 기록
- 02 starter와 수정본을 같은 요청으로 리뷰
- 03 올바른 지적·오탐·누락을 직접 판정

실제 관찰한 개선 한 가지 · 남은 오류 또는 판단 유보

실제 실행값의 전후 비교

```
comparison = [  
    ("before", before_receipt["result"]["payable_won"]),  
    ("after", after_receipt["result"]["payable_won"]),  
]  
expected = 3_000  
for version, actual in comparison:  
    print(version, actual, actual == expected)
```

Notebook이 실제 실행한 영수증 사용 / 수정 전 -2,000원 / 수정 후 3,000원 / 결과 숫자를 코드에 정답처럼 복사하지 않음

Codex 대화: 과잉 지적 개선

요청

solution의 코드를 같은 업무 규칙으로 다시 검토해 주세요.
이미 해결한 문제를 다시 지적하는지 확인하고,
실제 오류와 팀 스타일 제안을 분리해 주세요.
불확실하면 필요한 추가 근거를 명시해 주세요.

응답 확인 수정된 코드의 재검토 · 지적의 근거와 불확실성 · 새 테스트 필요 여부

하나의 점수보다 오류 분석

어떤 입력에서 왜 실패했는지 설명

- 테스트 누락인지 정책 누락인지 분류
- 잘못된 지적을 Prompt 예시로 보완
- 모델 교체 전 같은 사례로 재평가

본인 개선 기록

01 실패 입력과 수정 Diff 한 개 선택

02 Test 전후 결과 기록

03 다음 개선 항목 한 개 작성

개선 전·후 비교표 · 다른 사람이 재실행할 명령

3주차 8차시

16:40-17:30

Localhost와 다음 주 준비

서비스 시연과 GitHub 전달 준비

이론 10분 · 시연 8분 · 코드 실습 27분 · 확인 5분

사용 가능한 리뷰 서비스

입력부터 교정 결과까지 연결

- 사용자에게 Diff·정책 입력 제공
- 읽을 수 있는 리뷰와 Test 결과 표시
- 수정 후 같은 입력으로 다시 확인

수정 파일의 Localhost 연결

```
# Notebook 8차시에서 실행 명령 출력
print("python -m labs.day3.review_copilot.web",
      "--exercise-dir", EXERCISE_REL, "--port 8765")
```

```
# 출력한 명령을 저장소 루트 Terminal에서 실행
# 브라우저: http://127.0.0.1:8765/
# 종료: Terminal에서 Ctrl+C
```

--exercise-dir: 본인이 고친 폴더 연결 / 기본 폴더 실행과 Notebook 실행을 구별 / 같은 10,000 / 15,000 입력으로 재확인

장바구니 계산 화면

쿠폰 결제 서비스 리뷰

같은 입력으로 초안과 수정본의 실제 Python 코드를 실행합니다. 계산 → 실패 테스트 → Codex 리뷰 → 코드 수정 → 재검증 순서입니다.

상품 금액10000

쿠폰 금액15000

결제 예정액 비교

같은 테스트 실행

초안 · 수정 전

-2,000원

적용 할인 15,000원 · 배송비 3,000원

참고 구현 · 수정 후

3,000원

적용 할인 10,000원 · 배송비 3,000원

실제 Python 함수 실행 완료 · 실제 결제 없음

상품·쿠폰 입력과 결제 예정액 비교

같은 테스트의 전후 결과

초안 · 2/9 통과

bool 입력	× 실패
상품 금액을 넘는 쿠폰	× 실패
소수 금액	× 실패
무료 배송 기준 금액	✓ 통과
음수 쿠폰	× 실패
음수 상품 금액	× 실패
일반 쿠폰	✓ 통과
영수증 할인액	× 실패
할인 후 배송비	× 실패

▶ 실제 테스트 출력

참고 구현 · 9/9 통과

bool 입력	✓ 통과
상품 금액을 넘는 쿠폰	✓ 통과
소수 금액	✓ 통과
무료 배송 기준 금액	✓ 통과
음수 쿠폰	✓ 통과
음수 상품 금액	✓ 통과
일반 쿠폰	✓ 통과
영수증 할인액	✓ 통과
할인 후 배송비	✓ 통과

같은 테스트 · 초안 2/9 통과 → 참고 구현 9/9 통과

Codex의 근거와 수정 제안

[P1] 쿠폰이 상품 금액을 초과하면 결제 금액이 음수가 됩니다

`checkout.py:5`

`payable(10\_000, 15\_000)`이 요구값 0이 아니라 -5,000을 반환합니다. 따라서 동일 입력의 영수증 최종 결제 예정액도 3,000원이 아닌 -2,000원이 됩니다.

수정 제안: 실제 할인액을 `min(total\_won, coupon\_won)`으로 제한한 뒤 상품 금액에서 차감하세요.

실제 Codex 지적 · `checkout.py:5` · 쿠폰 상한 누락

화면에서의 전체 실행

01 결제 예정액 비교·같은 테스트 실행 버튼 확인

02 Codex CLI 리뷰 실행 후 Provider와 지적 확인

03 사람 결정은 Notebook 6차시에서 선택·재개

화면: 계산·Test·Markdown 리뷰 · Notebook: 선택을 반영한 review_report.md

Codex 대화: 서비스 화면

요청

리뷰 입력과 Markdown 결과를 보여주는 로컬 화면을 만들어 주세요.
실행 중·완료·오류 상태가 구별되어야 합니다.
잘못된 입력이면 오류 이유와 복구 방법을 보여 주세요.
Python 테스트와 로컬 접속 명령을 함께 제공해 주세요.

응답 확인 사용자가 눌러야 할 버튼 · 오류 후 다시 실행 · 코드·실행 방법의 일치

Git의 전달 단위

단위	의미	학생이 확인할 것
Branch	본인 변경의 작업 흐름	현재 branch 이름
Commit	하나의 목적을 기록한 변경	코드와 관련 Test
PR	변경을 함께 검토하는 요청	목표·Diff·Test 결과
Review	반영 전 검토 의견	수정 또는 반영하지 않을 이유

GitHub용 서비스 폴더

- 01 본인 저장소에 my-review-service 폴더 생성
- 02 수정한 starter의 checkout.py·checkout_checks.py 복사
- 03 그 폴더에서 python checkout_checks.py 실행

Notebook의 임시 출력과 별도 서비스 코드 · 복사한 코드에서도 같은 Test 통과

작은 Commit과 Draft PR

```
git switch -c codex/my-review-service
git status --short
git add my-review-service/checkout.py \
    my-review-service/checkout_checks.py
git diff --cached
git commit -m "fix: correct coupon calculation"
git push -u origin HEAD
gh pr create --draft
```

- 앞 장표의 파일 복사와 Test 완료 후 실행
- 학습용 본인 저장소
·branch 확인
- PR은 다음 주 자동 리뷰의 입력

3주차와 4주차의 연결

오늘의 로컬 검토

- 리뷰할 코드와 업무 규칙
- 실제 계산·Test·MD 리뷰
- 검토한 수정 Diff

다음 주 GitHub 자동화

- PR에서 Diff 수집
- 리뷰를 PR 줄에 연결
- 실행 이벤트와 중복 게시 관리

개인 서비스의 시작점

01 본인 작업을 Commit 단위로 정리

02 README에 실행 명령 작성

03 다른 Terminal에서 다시 시작

새 실행에서도 같은 기능 · 다음 주 사용할 PR 또는 Diff

개선 기록의 네 항목

문제·변경·실행 결과·남은 과제

- 어떤 입력이 왜 실패했는가
- 코드 또는 Prompt의 무엇을 바꿨는가
- Test와 실제 화면에서 무엇이 달라졌는가

3주차 8차시 · 16:40-17:30

2주 뒤의 개인 미니 프로젝트

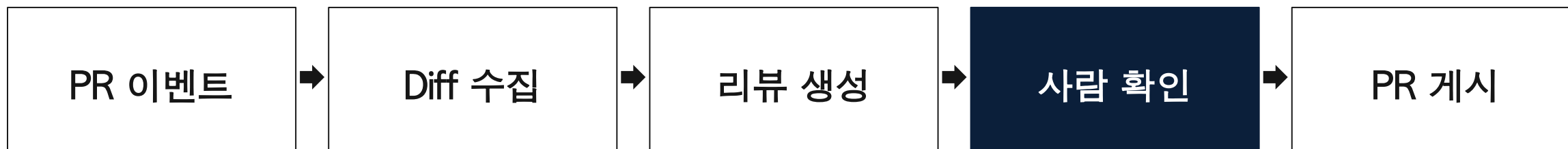
배운 기능을 본인 업무 서비스로 확장

- 5주차 15:00~18:00
- 제작 150분 + 휴식·Q&A 30분
- 개선 전후 기록 또는 짧은 화면 영상

4주차 수업 계획

차시	핵심 내용	직접 제작
1~2차시	GitHub PR 구조와 인증	본인 PR·Diff 수집 코드
3~4차시	라인 리뷰와 게시	리뷰 위치 매핑·미리보기
5~6차시	이벤트 자동화와 실패 처리	Actions·재시도·중복 방지
7~8차시	회의·리뷰 문서 연동	MD 템플릿·승인 후 게시

4주차 GitHub 자동 리뷰



문제	구현할 처리
중복 이벤트	PR·Commit·리뷰 버전 기준 중복 방지
API 오류	재시도 횟수·대기 시간·실패 기록
권한과 게시	최소 권한·게시 미리보기·승인 확인

5주차 수업 계획

차시	핵심 내용	직접 제작
1~2차시	회의록·리뷰 보고서 통합	업무 리포트 서비스
3차시	Workflow 통합	입력별 처리 경로
4~5차시	운영·관찰·배포 준비	오류 복구·LangSmith·Test
6~8차시	개인 미니 프로젝트	본인 서비스와 개선 기록

미니 프로젝트 3시간 운영

시간	작업	확인 결과
15:00-15:25	문제와 최소 기능 선택	입력·사용자·기대 결과
15:25-16:15	Codex 대화로 구현	실행 가능한 핵심 기능
16:15-17:00	실패 재현과 개선	Test·수정 전후 비교
17:00-17:30	실행 안내·결과 정리	README·화면·개선 기록
17:30-18:00	후반 휴식·Q&A	개별 질문·선택 공유

미니 프로젝트 선택 예시

서비스 확장

- 회의 To-do 담당자 확인
- PR 리뷰와 수정 요청 정리
- 문서의 누락 항목 검사

작은 범위의 완성

- 입력 한 종류
- 핵심 동작 한 가지
- 실패 Test와 개선 한 가지

개선 전후의 공유 양식

항목	짧게 설명할 내용
문제	누가 어떤 입력에서 불편했는가
구현	어떤 함수·화면·도구를 만들었는가
개선	코드·Prompt·Workflow의 변경
검증	같은 입력의 전후 결과와 Test

3주차 8차시 · 16:40-17:30

Q&A와 실습 복구

17:30~18:00

- 실행되지 않는 첫 오류
- 수정 전후 결과의 차이
- 다음 주 PR 준비